

C# Synchronization Primitives Comparison				
Primitive	Scope	Use Case	Performance	Notes
lock	In-process	Simple critical section	Fast (user-mode)	Syntactic sugar for Monitor. Only works with In-process scope.
Monitor	In-process	Advanced locking (Wait/Pulse)	Fast	More control than lock, supports Wait/Pulse/Notify methods.
Mutex	Cross-process	Synchronize across processes	Slower (kernel-mode)	OS-level object; use when multiple processes need synchronization.
Semaphore	In-process / Cross	Limit concurrent threads (e.g., max N)	Medium	Good for throttling access to limited resources.
SemaphoreSlim	In-process	Lightweight semaphore	Faster than Semaphore	Preferred in-process; async-friendly APIs (WaitAsync, Release).
ReaderWriterLockSlim	In-process	Many readers, few writers	Medium	Optimized for read-heavy workloads; upgrade to write access.
Interlocked	In-process	Atomic operations (increment, exchange)	Very fast	Best for counters/flags; no blocking; per-variable.
SpinLock	In-process	Very short critical sections	Fast if short	Busy-wait; avoid if lock may be held long; can starve.
Barrier	In-process	Sync multiple threads at a phase	Medium	Useful for phased parallel algorithms.
CountDownEvent	In-process	Wait until N signals occur	Medium	Latch-like; coordinate completion of N operations.
ResetEvent	In-process	Thread signaling (manual open/close)	Medium	Stays signaled until reset; gate for many threads.
AutoResetEvent	In-process	Thread signaling (auto-close)	Medium	Releases one waiter then resets automatically.